

Memoria de análisis y diseño

Base de datos de NexShop Group S.A.

Autor: Samuel Pérez-Herrero Soler

Proyecto: Diseño e implementación de la base de datos corporativa

Alcance: catálogo, ventas online y presenciales, logística, atención al cliente, fidelización

Resultado: 25 tablas base + 1 vista derivada, script SQL validado en MySQL/MariaDB

1. Contexto y objetivo

NexShop Group S.A. es una empresa de distribución y venta minorista fundada en 2015 con sede en Valencia. Opera una tienda online (nexshop.es) y tres tiendas físicas (Valencia, Madrid y Barcelona), apoyadas por un almacén central, con una plantilla en torno a 47 personas. El objetivo de este proyecto es diseñar desde cero una base de datos relacional que soporte su operativa: gestión del catálogo, ventas por los dos canales, aprovisionamiento, control de inventario entre sedes, atención al cliente, valoraciones de producto y el programa de puntos de fidelización.

Esta memoria documenta y **justifica** cada decisión de modelado. Para cada entidad se indica si responde a un requisito explícito del enunciado (**[Cliente]**) o si es una propuesta de diseño propia para resolver bien el problema (**[Propio]**).

2. Metodología y criterios de diseño

- Se parte del texto del enunciado, se subrayan los sustantivos relevantes (candidatos a entidad) y los verbos (candidatos a relación) y se contrastan con las preguntas de negocio que la base debe responder.
- Se aplica normalización hasta 3FN: cada hecho se almacena una sola vez y los atributos dependen únicamente de la clave. Las redundancias que se conservan (p. ej. producto.pvp) son decisiones conscientes de rendimiento, justificadas más abajo.
- Toda relación muchos-a-muchos se resuelve con una tabla intermedia.
- La información que cambia con el tiempo y debe conservarse (precios, condiciones de proveedor, puntos) se modela con filas de vigencia temporal, no sobrescribiendo el dato anterior.
- La integridad se delega en el motor siempre que es posible: claves primarias y ajenas, restricciones UNIQUE, CHECK y tipos ENUM para los dominios cerrados.

3. Catálogo de entidades

El modelo consta de 25 tablas. La siguiente tabla resume cada una con su justificación y origen.

Entidad	Para qué sirve	Origen
categoría	Nivel superior del catálogo (p. ej. Informática).	[Cliente]
subcategoría	Agrupar productos dentro de una categoría (p. ej. Portátiles). Un producto pertenece a una sola.	[Cliente]
producto	Artículo del catálogo. Pertenecer a una subcategoría.	[Cliente]
histórico precio	Registro de todos los PVP que ha tenido un producto (auditoría de precios).	[Cliente]
promoción	Campaña de descuento porcentual con rango de fechas.	[Cliente]
promoción producto	Puente N:M entre promociones y productos a los que aplican.	[Propio]
proveedor	Proveedor de productos, con su representante comercial de NexShop.	[Cliente]
suministro	Resuelve el N:M producto-proveedor con precio de coste, plazo e histórico de condiciones.	[Cliente]
ubicación	Sede física con stock propio: tiendas y almacén central.	[Cliente]
empleado	Trabajador asignado a una sede, con su puesto (vendedor, encargado, agente, comercial...).	[Cliente]
cliente	Cliente registrado (o creado al vincular compras presenciales).	[Cliente]
dirección	Direcciones del cliente (domicilio, trabajo...). Un cliente puede tener varias.	[Cliente]
pedido	Pedido del canal online. Requiere cliente y dirección de envío.	[Cliente]
línea pedido	Detalle del pedido: producto, cantidad y precio congelado.	[Cliente]
envío	Cada uno de los envíos que genera un pedido (un pedido puede tener varios).	[Cliente]
línea envío	Qué productos/cantidades van en cada envío parcial.	[Propio]
ticket venta	Venta presencial en tienda. Cliente opcional (ventas anónimas).	[Cliente]
línea ticket venta	Detalle de la venta presencial.	[Cliente]

Entidad	Para qué sirve	Origen
devolución	Documento de devolución presencial, ligado al ticket original.	[Cliente]
línea devolución	Productos y cantidades devueltos.	[Propio]
incidencia	Ticket de atención al cliente; las devoluciones online se gestionan aquí.	[Cliente]
stock	Existencias de cada producto en cada ubicación (N:M con cantidad).	[Cliente]
transferencia	Movimiento de stock entre sedes, con empleado que lo autoriza.	[Cliente]
valoración	Puntuación y comentario de un cliente sobre un producto (una por par).	[Cliente]
movimiento puntos	Histórico de puntos ganados y canjeados; fuente del saldo.	[Cliente]
v saldo puntos	Vista que calcula el saldo de puntos por cliente desde el histórico.	[Propio]

4. Atributos y decisiones de tipo destacadas

- Importes en DECIMAL(10,2) (nunca FLOAT, para no perder precisión en dinero) y porcentajes en DECIMAL(5,2).
- Dominios cerrados con ENUM: estado de pedido, estado de envío, tipo de ubicación, puesto del empleado, tipo de incidencia, tipo de movimiento de puntos.
- Precio congelado en líneas de pedido y de venta (precio_unitario): el documento conserva el precio del momento aunque el PVP cambie después.
- Nulos con significado: cliente.email nulo = aún no registrado online; ticket_venta.id_cliente nulo = venta anónima; fecha_fin nula en los históricos = condición vigente.

5. Relaciones y cardinalidades

Las relaciones 1:N principales son: una categoría tiene muchas subcategorías; una subcategoría agrupa muchos productos; un cliente tiene muchas direcciones y realiza muchos pedidos; un pedido se compone de muchas líneas y genera muchos envíos; una ubicación tiene muchos empleados, mucho stock y origina muchos envíos y transferencias; un empleado representa a varios proveedores, atiende muchas incidencias y realiza muchas ventas. Las relaciones N:M se detallan en el apartado siguiente.

6. Resolución de las relaciones muchos-a-muchos

- **producto** ↔ **promoción** → promocion_producto (puente simple).

- **producto** ↔ **proveedor** → suministro. No es un puente simple: lleva atributos propios (precio de coste, plazo) e historial temporal de las condiciones pactadas.
- **producto** ↔ **ubicación** → stock, con la cantidad como atributo de la relación.
- **documento** ↔ **producto** (pedido, envío, ticket, devolución) → sus respectivas tablas de líneas, cada una con clave compuesta y cantidad.
- **cliente** ↔ **producto** → valoración, con UNIQUE(id_cliente,id_producto) para garantizar una sola valoración por par.

7. Gestión de históricos temporales

Tres requisitos exigen conservar cómo cambian los datos con el tiempo. Se resuelven con el mismo patrón: una fila por periodo de vigencia, con fecha_inicio y fecha_fin, donde la fila vigente es la que tiene fecha_fin = NULL:

- historico_precio para los PVP del producto.
- suministro para las condiciones de cada proveedor.
- movimiento_puntos para el saldo de fidelización (cada movimiento con su signo).

8. Integridad y reglas de negocio en el esquema

- Claves ajenas en todas las relaciones; borrado en cascada solo en las líneas de detalle respecto de su documento.
- CHECK: cantidades > 0, precios y puntos ≥ 0, puntuación entre 1 y 5, descuentos entre 0 y 100, fecha_fin ≥ fecha_inicio, origen ≠ destino en transferencias, y una incidencia resuelta obliga a tener fecha de cierre.
- UNIQUE: DNI y email corporativo de empleado, email de cliente, número de seguimiento de envío, y el par cliente-producto de las valoraciones.

9. Preguntas de reflexión

9.1. Un pedido online puede dar lugar a varios envíos. ¿Cómo lo has modelado?

Con una relación 1:N entre pedido y envío: un pedido tiene uno o varios envíos y cada envío pertenece a un único pedido. Cada envío guarda su número de seguimiento, transportista, fecha estimada y la **ubicación de origen**, de modo que distintos envíos del mismo pedido pueden salir de almacenes diferentes. Para saber qué se manda en cada uno añado `linea_envio` (qué productos y cantidades viajan en ese envío), lo que permite representar **envíos parciales**: la suma de cantidades de un producto en los envíos de un pedido debe coincidir con lo pedido en `linea_pedido`. Además, `envio.tipo` distingue entrega de recogida, con lo que una devolución online se representa como un envío de tipo recogida.

9.2. Las ventas presenciales y los pedidos online, ¿una sola tabla o dos?

He optado por **dos modelos separados**: `pedido/ticket_venta/NOT NULL`. La contrapartida es que una consulta de “ventas totales de un producto por ambos canales” requiere un UNION; lo asumo y, si se usara mucho, se encapsularía en una vista. Gano claridad e integridad, que es lo prioritario aquí.

9.3. ¿Es lo mismo el histórico de precios que el histórico de promociones?

No, son cosas distintas y por eso las modelo por separado. El **histórico de precios** (`historico_precio`) registra la evolución del **PVP base** del producto: cada fila es un periodo en el que el producto tuvo ese precio. Una **promoción** (`promocion + promocion_producto`) es un **descuento porcentual temporal** que se aplica sobre un conjunto de productos durante un rango de fechas y que **no modifica** el PVP base. Separarlos permite que coexistan (un producto puede estar de promoción mientras su precio base sigue siendo X) y que cada uno conserve su propio historial sin contaminar al otro.

9.4. El saldo de puntos se calcula desde el histórico. ¿Guardarías además un campo “saldo actual”?

He decidido **no** guardarlo. La tabla `movimiento_puntos` es la única fuente de verdad y el saldo se deriva sumando cada movimiento con su signo (lo encapsulo en la vista `v_saldo_puntos`). Guardar un `saldo_actual` redundante introduce el riesgo clásico de incoherencia: si un movimiento se inserta, corrige o anula y el campo no se actualiza, el saldo “oficial” mentiría. Como el propio requisito pide que el saldo siempre pueda calcularse desde el histórico, un valor derivado lo cumple por construcción. El precio es que cada consulta del saldo hace una agregación; a gran escala podría cachearse el saldo en una columna mantenida por triggers o un proceso periódico, pero eso reintroduce la posibilidad de descuadre, así que para este alcance prefiero el saldo calculado.

9.5. Un cliente que compra en tienda sin cuenta puede luego vincularse a una cuenta online. ¿Cómo lo has previsto?

La venta presencial admite cliente **opcional**: ticket_venta.id_cliente es una clave ajena que puede ser NULL (venta anónima, que es lo habitual cuando no tenemos datos de la persona). Cuando ese cliente se registra online más tarde y aporta sus tickets, se **vinculan** sus compras actualizando esas filas (UPDATE ticket_venta SET id_cliente = ... WHERE id_ticket_venta = ...) para que apunten a su nueva cuenta. Además, cliente permite email y password nulos, de forma que una fila puede representar a alguien conocido en tienda pero todavía no plenamente registrado. Para que el enlace sea fiable, el ticket lleva impreso su identificador (podría añadirse un código), que el cliente presenta. Así cubro tanto la venta anónima como la atribución posterior sin necesidad de una tabla de “clientes anónimos”.

9.6. ¿Hay algún requisito que no hayas implementado o que hayas simplificado?

Sí, y los dejo explícitos para no perder puntos por omisión silenciosa:

- **Promoción de cumpleaños:** guardo cliente.fecha_nacimiento, que es el dato necesario, pero el envío de la promoción es un proceso de la aplicación, no una entidad de datos, así que no lo modelo como tabla.
- **Encargado de tienda:** lo represento como el empleado con puesto = 'encargado' asignado a esa ubicación, en lugar de una FK ubicacion.id_encargado, para evitar una dependencia circular entre ubicacion y empleado. La regla “exactamente un encargado por tienda” queda como norma de negocio, no forzada por el esquema.
- **Transferencias de un solo producto:** el enunciado habla de “producto y cantidad” en singular, así que transferencia mueve un producto por registro; una transferencia multi-línea sería una extensión (cabecera + líneas) análoga a los demás documentos.
- **Pagos y facturación** (método de pago, facturas) no se modelan porque el enunciado no los pide.
- **Coherencia cantidades pedido/envío** (que lo enviado sume lo pedido) es una regla de negocio que no se impone con una restricción de tabla.

10. Conclusión

El modelo resultante cubre todos los requisitos del enunciado con 25 tablas normalizadas, resuelve correctamente las relaciones N:M y los históricos temporales, y delega la integridad en el motor mediante claves, UNIQUE, CHECK y ENUM. El script schema.sql se ha cargado sin errores en MariaDB 10.11, los datos de prueba de datos.sql ejercitan todos los casos límite (nulos, ventas anónimas, multi-envío, históricos) y la batería de consultas.sql se ejecuta correctamente, incluida la vista de saldo de puntos.